

tree_clustering.py

User Guide and Algorithm Explanation

Cursor (Composer)

under instruction from Hugh Murrell

August 10, 2026

Abstract

`tree_clustering.py` is a command-line batch tool that mirrors the k-DPIs clustering workflow of the aliViz web application. For each Newick phylogeny in an input directory it parses days-post-infection (DPI) from the filename, groups tip names, selects a founder tip from the smallest group id, reroots and ladderizes the tree, runs automatic k-DPIs bipartition clustering, and writes an annotated linear SVG tree plot. Optionally it annotates a matching FASTA alignment. After processing a directory it writes a housekeeping CSV and a pie-chart SVG summarising cluster-count distributions across CAP samples. This document is the complete user guide and a detailed description of the algorithms implemented in the script.

Contents

1	Introduction	3
1.1	Purpose	3
1.2	Relationship to aliViz	3
1.3	Requirements	3
2	Command-line usage	3
2.1	Synopsis	3
2.2	Positional arguments	3
2.3	Optional arguments	3
2.4	Examples	4
3	Inputs and outputs	4
3.1	Tree filenames and DPI	4
3.2	Tip name grouping	4
3.3	Per-tree outputs	5
3.4	Batch housekeeping outputs	5
4	Pipeline overview	5
5	Tree geometry primitives	6
5.1	Path distance	6
5.2	Cluster depth (LCA height)	6
5.3	Mean pairwise distance and DPI	6
5.4	Expected early tip distance	6
6	DPI from filename	7
7	Grouping	7

8 Founder selection	7
9 Reroot on founder	7
10 Ladderize by depth	7
11 k-DPIs Auto clustering	8
11.1 Pre-noise (long branches)	8
11.2 Bubble tree	8
11.3 Candidate bipartitions	8
11.4 Which leaf to split (Auto)	8
11.5 Auto loop	8
12 Cluster assignment and noise	8
13 Optional FASTA annotation	9
14 SVG export	9
14.1 Layout	9
14.2 Scale modes	9
14.3 Per-cluster target lines	9
14.4 Colours and legend	10
14.5 Example plots	10
15 Housekeeping pie chart	11
16 Default parameters (summary)	11
17 Failure modes and exit codes	11
18 Worked numerical sketch	11
19 File map	12
20 Versioning note	12
A Colour palette (clusters)	12

1 Introduction

1.1 Purpose

The script automates the same scientific workflow that a user would perform interactively in aliViz:

1. load a phylogenetic tree;
2. group sequences by a name field;
3. reroot on a founder;
4. ladderize;
5. run k-DPIs Auto clustering;
6. export a DPI-aware tree SVG (and optionally annotate FASTA).

It is intended for batch analysis of many participant trees (typically CAP cohort Newick files) with consistent parameters.

1.2 Relationship to aliViz

Behaviour is intended to match aliViz where practical:

- same default grouping (delimiter `_`, field 3);
- same k-DPIs bipartition objective ($\min \max(\text{depth}_1, \text{depth}_2)$) with both children required to meet minimum cluster size;
- same DPI estimator from mean pairwise tree-path distance;
- same cluster / noise naming convention (`_cl-n` / `_cl-n`);
- SVG layout inspired by aliViz linear AUTO / DPI export.

One intentional difference: founder selection uses the *tree-path medoid* of the group with the smallest group id (not Hamming distance on an alignment).

1.3 Requirements

- Python 3.9+ (uses `from __future__ import annotations`);
- standard library only (no Biopython or other third-party packages).

2 Command-line usage

2.1 Synopsis

```
python3 tree_clustering.py INPUT_DIR OUTPUT_DIR [options]
```

2.2 Positional arguments

INPUT_DIR Directory containing Newick tree files. Recognised extensions: `.nwk`, `.newick`, `.tree`, `.tre`, `.treefile`, `.txt`.

OUTPUT_DIR Directory for SVG trees, optional annotated FASTA, `housekeeping.csv`, and `housekeeping.svg`. Created if missing.

2.3 Optional arguments

Option	Default	Meaning
<code>--delimiter</code>	<code>_</code>	Character splitting tip names into fields for grouping.

Option	Default	Meaning
<code>--field</code>	3	1-based field index used as the group label (aliViz Group dialog).
<code>--dsr</code>	5×10^{-5}	Daily substitution rate (mutations / site / day).
<code>--min-size</code>	1	Minimum allowed size of each child after a bipartition. Singletons are still reclassified as noise at assignment.
<code>--max-splits</code>	63	Maximum number of Auto splits (upper bound on $k - 1$).
<code>--scale</code>	auto	SVG branch scale: auto (fit tree + target marks) or dpi (legacy: expected depth at 520/16 px).
<code>--fasta-dir</code>	(none)	Directory of FASTA files with the same basename as each tree. When set, tip names and FASTA headers are annotated with cluster suffixes.
<code>--no-long-branch-noise</code>	off	Disable removal of tips whose incoming branch exceeds $\text{etd} = 2 \times \text{dsr} \times \text{DPI}$.

2.4 Examples

```
# Minimal batch run
python3 tree_clustering.py trees_in/ trees_out/

# Match aliViz grouping and annotate FASTA
python3 tree_clustering.py trees_in/ trees_out/ \
  --delimiter _ --field 3 \
  --fasta-dir fasta_in/

# Legacy DPI 1/16 scale, no long-branch pre-noise
python3 tree_clustering.py trees_in/ trees_out/ \
  --scale dpi --no-long-branch-noise
```

3 Inputs and outputs

3.1 Tree filenames and DPI

DPI days are parsed from the tree basename with the regular expression

$$\text{dpi} [+-] (+) \text{---}$$

(case-insensitive). If several matches occur, the *largest* integer is used. If none are present, DPI defaults to 14.

Examples:

- C002_CAP008_dpi+79.nwk $\rightarrow$ DPI = 79;
- x_dpi-14_dpi+100.nwk $\rightarrow$ DPI = 100;
- plain.nwk $\rightarrow$ DPI = 14.

CAP identifiers for housekeeping are parsed with (CAP\d+) (e.g. CAP008).

3.2 Tip name grouping

Tip names are split on `--delimiter`. Field `--field` (1-based) becomes the group label. Existing cluster suffixes of the form `_cl-n`, `_cl-na`, or `_cl-n` are stripped before grouping.

Example (delimiter `_`, field 3):

CAP008_2000_G1_a $\rightarrow$ group G1.

3.3 Per-tree outputs

For a tree with stem S , G groups and C non-noise clusters, the script writes

$S_gr-G_cl-C.svg$

into OUTPUT_DIR.

If `--fasta-dir` is set, a matching FASTA ($S.fasta$ / $.fa$ / $.fas$ / $.fna$) is required. An annotated copy is written to OUTPUT_DIR under the same filename, with headers ending in $_cl-n$ or $_cl-n$ (noise). Tree tip labels in the SVG are annotated the same way when FASTA annotation is enabled.

3.4 Batch housekeeping outputs

After all trees are processed:

`housekeeping.csv`

Columns **CAPid**, **Groups**, **Clusters**; one row per successfully processed tree; sorted by numeric CAP id.

`housekeeping.svg`

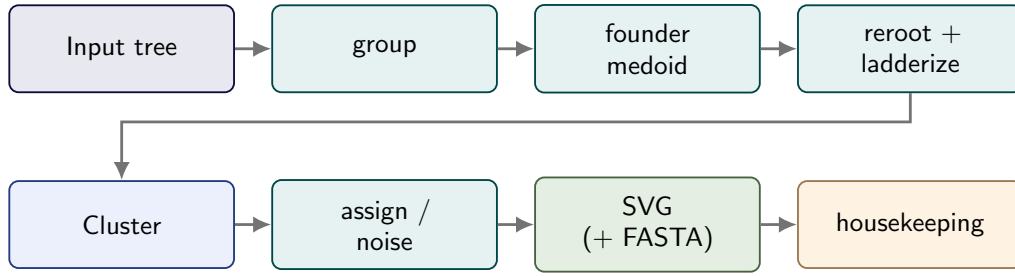
Pie chart of CAP samples binned by cluster count: 1, 2, 3–4, 5–8, 9–16, ... (empty bins omitted).

4 Pipeline overview

For each tree file the script executes the following stages in order.

1. **Parse** Newick into an in-memory tree; set parent pointers.
2. **Parse DPI** from the filename (Section 6).
3. **Group** tip names (Section 7).
4. **Founder**: take the group with the smallest group id; choose the tree-path medoid (Section 8).
5. **Reroot** on the founder (Section 9).
6. **Ladderize** by subtree depth, ascending (Section 10).
7. **Layout** assign root-to-node depths and tip row indices.
8. **k-DPIs Auto** clustering (Section 11).
9. **Assign** cluster IDs; reclassify singletons as noise; number non-noise clusters 1.. k by tip order (Section 12).
10. **Optional FASTA / tip annotation** (Section 13).
11. **SVG export** with AUTO or DPI scale and per-cluster target lines (Section 14).

Finally, write housekeeping CSV and pie chart.

Figure 1: High-level data flow of `tree_clustering.py`.

5 Tree geometry primitives

Let each edge carry a non-negative length $\ell(u)$ stored as the *incoming* length of child u . The root-to-node distance $d(u)$ satisfies

$$d(\text{root}) = 0, \quad d(u) = d(\text{parent}(u)) + \ell(u).$$

(The implementation includes $\ell(\text{root})$ in $d(\text{root})$ if present; Newick roots usually have $\ell = 0$.)

5.1 Path distance

For tips a, b with lowest common ancestor $\text{lca}(a, b)$,

$$\delta(a, b) = d(a) + d(b) - 2 d(\text{lca}(a, b)).$$

5.2 Cluster depth (LCA height)

For a tip set T with $|T| \geq 2$ and $L = \text{lca}(T)$,

$$\text{depth}(T) = \max_{t \in T} (d(t) - d(L)).$$

For $|T| < 2$, depth is defined as 0.

5.3 Mean pairwise distance and DPI

$$\bar{\delta}(T) = \frac{2}{|T|(|T| - 1)} \sum_{a < b} \delta(a, b) \quad (|T| \geq 2).$$

Days-post-infection for a tip set is

$$\widehat{\text{DPI}}(T) = \text{round} \left(\frac{\bar{\delta}(T)}{2 \text{ dsr}} \right). \quad (1)$$

This matches the aliViz tree-based DPI estimator ($\text{dpi} = \bar{\delta} / (2 \times \text{dsr})$).

5.4 Expected early tip distance

Under a strict-clock star model,

$$\text{etd} = 2 \times \text{dsr} \times \text{DPI}. \quad (2)$$

Long-branch pre-noise removes tips whose incoming length exceeds etd. The SVG / scale-bar *target length* used for vertical marks is the one-sided quantity

$$\tau = \text{DPI} \times \text{dsr} \quad (3)$$

(half of etd), i.e. expected root→tip depth for DPI days.

6 DPI from filename

Let M be the set of integers matched by `dpi[+|-]N` in the filename. Then

$$\text{DPI} = \begin{cases} \max M & M \neq \emptyset, \\ 14 & \text{otherwise.} \end{cases}$$

This DPI is used as:

- the Auto split threshold (split only while $\widehat{\text{DPI}} > \text{DPI}$);
- the long-branch threshold via (2);
- the SVG target length τ via (3).

7 Grouping

For each tip name s :

1. strip a trailing cluster suffix matching `delimiter cl-(digits|na|n)`;
2. split on `delimiter`;
3. if field `field` exists and is non-empty, use it as the group label.

Group IDs are contiguous integers $0, 1, \dots$ ordered by sorted unique labels. Tips lacking the field are excluded from grouping (and therefore cannot be founders).

8 Founder selection

Let G^* be the group with the smallest group id (group ids are unique, so this choice is unambiguous). Among tips T^* of G^* , the founder f is the *tree-path medoid*

$$f = \arg \min_{a \in T^*} \sum_{b \in T^*} \delta(a, b). \quad (4)$$

If $|T^*| = 1$, that tip is the founder.

9 Reroot on founder

Rerooting places the founder as an outgroup child of a new root:

1. Walk the path from founder leaf to the old root.
2. Detach the founder from its parent; let ℓ_f be the founder edge length.
3. Invert the remaining spine so the former parent becomes the root of the rest of the tree, transferring edge lengths along the flipped path.
4. Create a new root with children $\{\text{founder}, \text{rest}\}$ and set both edge lengths to $\ell_f/2$ (midpoint of the founder edge).

Pairwise tip path distances are preserved under this reroot (undirected metric).

10 Ladderize by depth

For each node u , define subtree max depth recursively:

$$D(u) = \begin{cases} \ell(u) & u \text{ leaf,} \\ \ell(u) + \max_{c \in \text{children}(u)} D(c) & \text{otherwise.} \end{cases}$$

Children of every internal node are then sorted by ascending $D(c)$ (“shallow subtrees first”), matching aliViz depth ladderize mode. Tip display order (top to bottom) follows this child order after layout.

11 k-DPIs Auto clustering

11.1 Pre-noise (long branches)

Unless `--no-long-branch-noise` is set, tips with $\ell(t) > \text{etd}$ are removed from clustering and later labelled noise. If all tips would be removed, the filter is abandoned and all tips are kept.

11.2 Bubble tree

Clustering maintains a binary partition tree of tip sets (“k-DPIs leaves”). Each atomic node stores tip set T , $\widehat{\text{DPI}}(T)$ from (1), and $\text{depth}(T)$.

Initially there is one leaf containing all (non-pre-noise) tips.

11.3 Candidate bipartitions

To split a tip set T with $|T| \geq 2m$ where $m = \text{min-size}$:

1. For every phylogeny node, let R be the subset of T descending from it.
2. If $0 < |R| < |T|$, consider the bipartition $(L, R) = (T \setminus R, R)$.
3. Accept the candidate only if $|L| \geq m$ and $|R| \geq m$ (no undersized / noise-only children).
4. Score = $\max(\text{depth}(L), \text{depth}(R))$.
5. Choose the bipartition with minimal score.

If no candidate exists, a fallback enumerates nonempty proper unions of children of $\text{lca}(T)$ (for polytomies with 2–20 children).

11.4 Which leaf to split (Auto)

Among atomic leaves with size $\geq 2m$, not marked unsplittable, and with $\widehat{\text{DPI}} > \text{DPI}_{\text{target}}$, Auto selects the leaf with largest DPI (ties: larger size).

11.5 Auto loop

Repeat until no eligible leaf remains or `max-splits` is reached:

1. find leaf to split;
2. apply best bipartition (or mark unsplittable and continue);
3. count one successful split.

12 Cluster assignment and noise

After Auto finishes, atomic leaves are classified as follows:

1. Leaves with $|T| < m$ become noise.
2. **Singleton reclassification:** every remaining leaf with $|T| = 1$ is reclassified as noise *before* ID assignment (even when $m = 1$).
3. Remaining non-noise leaves are sorted by the minimum tip-row index of their members (top of the ladderized tree first).
4. They receive contiguous IDs $1, 2, \dots, k$.
5. Pre-noise tips and all noise leaves map to cluster id -1 .

Display / FASTA suffixes:

- noise (id = -1): `_cl-n`
- cluster $n \geq 1$: `_cl-n`

13 Optional FASTA annotation

When `--fasta-dir DIR` is supplied:

1. Locate `DIR/SS$.{fasta,fa,fas,fna}` for tree stem S .
2. For each FASTA header, strip any old cluster suffix, look up the cluster id, and append `_cl-n` or `_cl-n`.
3. Write the annotated FASTA to `OUTPUT_DIR` (same basename).
4. Rename tree tip labels correspondingly so the SVG shows annotated names.

If no matching FASTA exists, that tree fails with an error (other trees continue).

14 SVG export

14.1 Layout

After clustering, tips are drawn top-to-bottom in ladderized order. An extra top spacer row holds the scale bar (analogous to aliViz placing the scale on the reference row above the first tip). Horizontal position of a node is

$$x = 5 + d(u) \cdot x_{\text{scale}}.$$

Branch region width is $W = 520$ px.

14.2 Scale modes

AUTO (default). Fit depth

$$D_{\text{fit}} = \max(D_{\text{tree}}, \tau, \max_c(d(\text{lca}_c) + \tau))$$

where the last term runs over non-noise clusters c (so every cluster target mark remains inside the plot). Then $x_{\text{scale}} = W/D_{\text{fit}}$. When DPI was detected in the filename, the scale-bar label is (N DPI @ `dsr`).

DPI (legacy, `--scale dpi`). Place τ at $W/16 = 32.5$ px: $x_{\text{scale}} = (W/16)/\tau$.

14.3 Per-cluster target lines

For each non-noise cluster c with tip set T_c and $L_c = \text{lca}(T_c)$, draw a *solid* vertical line at

$$x_c = 5 + (d(L_c) + \tau) x_{\text{scale}}$$

(clamped into the branch region if needed), spanning only the vertical extent of T_c 's tip rows, coloured with the cluster palette colour of c . No text annotation is attached to these lines. Tip diamonds use half the historical aliViz size (half-diagonal 5 px); target lines use stroke width 2.

14.4 Colours and legend

- Tip diamonds: cluster colour (noise grey $\blacksquare$ #9ca3af).
- Tip labels: group colour (founder highlighted magenta); optional [Founder] suffix.
- Legend: groups with $(n, \overline{\text{DPI}})$ and clusters with $(n, \overline{\text{DPI}})$, plus noise count.

14.5 Example plots

Figures 2 and 3 show real batch outputs produced by `tree_clustering.py` (AUTO scale). Both trees were rerooted on a founder, ladderized by depth, clustered with k-DPIs Auto, and drawn with per-cluster target bars and a group/cluster legend.

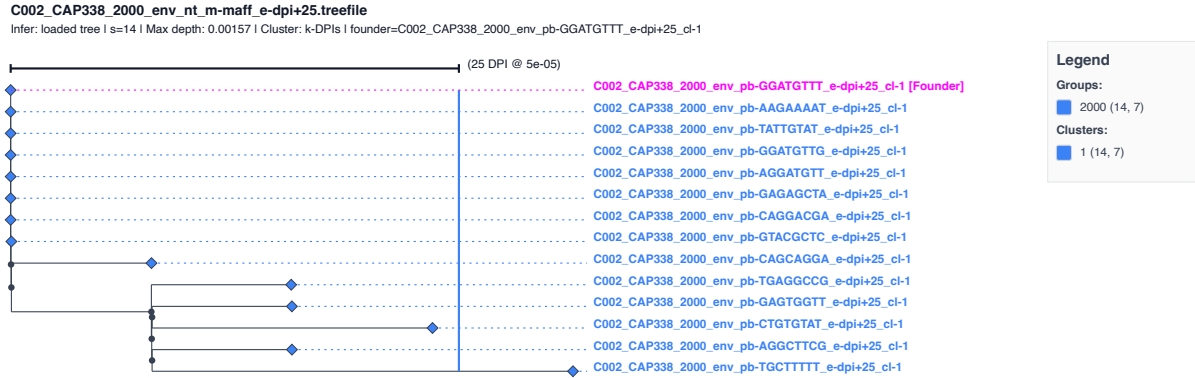


Figure 2: Single-cluster example (C002_CAP338_2000_...dpi+25_gr-1_cl-1.svg; $s=14$ tips, max depth ≈ 0.00157). Filename DPI is 25, with scale bar (25 DPI @ $5e-05$). All tips form one non-noise cluster, so diamonds share cluster colour 1. A solid vertical target bar in that colour is drawn at $\text{lca} + \tau$ (here the founder-rerooted root), spanning only the cluster tip rows. Tip labels use the group colour palette; the founder tip is marked [Founder] in magenta and sits as the outgroup after rerooting. The top spacer row holds the scale bar; the right-hand legend lists the single group as 2000 (n , DPI) and cluster 1 similarly. Noise tips (if any) appear grey and are listed under Noise.

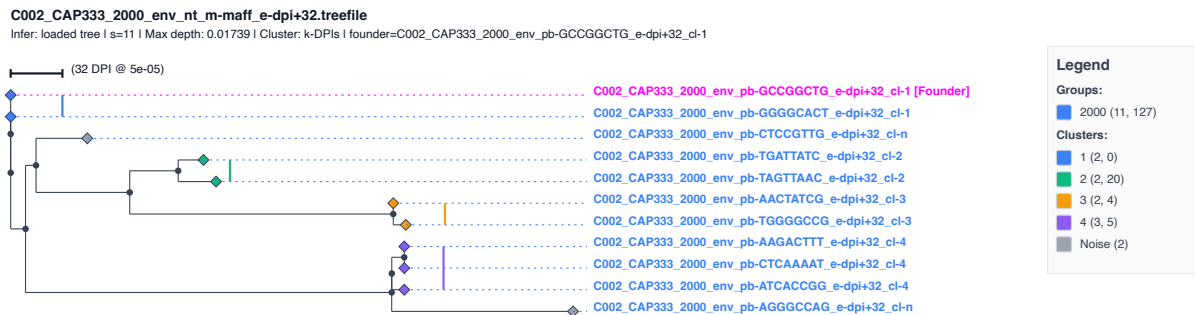


Figure 3: Multi-cluster example (C002_CAP333_2000_...dpi+32_gr-1_cl-4.svg; $s=11$ tips, max depth ≈ 0.0174). Filename DPI is 32, with scale bar (32 DPI @ $5e-05$). One time-point group is present (gr-1). k-DPIs Auto produced four non-noise clusters (cl-4) plus two noise tips: tip diamonds and solid target bars use palette colours 1–4; noise diamonds are grey. Each target bar sits at that cluster's $\text{lca} + \tau$ and covers only the vertical span of its tips—so later (deeper) clusters have marks further to the right. Tip name colours track the group; the founder is highlighted in magenta with [Founder]. Tip suffixes $\text{_cl-}n$ / $\text{_cl-}n$ match optional FASTA annotation.

15 Housekeeping pie chart

Each CAP sample contributes its non-noise cluster count C to a bin:

C	Bin label
≤ 1	1
2	2
3–4	3–4
5–8	5–8
9–16	9–16
$\vdots$	powers-of-two intervals

Only non-empty bins appear as pie slices, ordered by lower endpoint.

16 Default parameters (summary)

Parameter	Default
Daily substitution rate (dsr)	0.00005
DPI if absent from filename	14
Group delimiter / field	_ / 3
Min cluster size	1
Max Auto splits	63
SVG scale mode	auto
Branch plot width	520 px
Long-branch pre-noise	enabled

17 Failure modes and exit codes

- Per-tree failures (parse error, missing group field, missing FASTA when required, etc.) are reported on stderr; other trees continue.
- Exit code 0: all trees succeeded.
- Exit code 1: bad arguments / empty input directory.
- Exit code 2: at least one tree failed.

18 Worked numerical sketch

Suppose $\text{dsr} = 5 \times 10^{-5}$ and filename $\text{DPI} = 14$. Then

$$\tau = 14 \times 5 \times 10^{-5} = 7 \times 10^{-4}, \quad \text{etd} = 1.4 \times 10^{-3}.$$

A tip set with mean pairwise path distance $\bar{\delta} = 2.1 \times 10^{-3}$ has

$$\widehat{\text{DPI}} = \text{round}\left(\frac{2.1 \times 10^{-3}}{2 \times 5 \times 10^{-5}}\right) = \text{round}(21) = 21 > 14,$$

so Auto will attempt to split it (subject to size and bipartition constraints). After a successful split into children with $\text{DPI} \leq 14$, Auto stops on those leaves.

19 File map

Path	Role
tree_clustering.py	Implementation (this guide)
aliViz.html	Interactive reference UI
latex/tree_clustering_guide.tex	This document
latex/images/	Example SVG/PDF figures
latex/Makefile	Build PDF via <code>pdflatex</code>

20 Versioning note

Defaults and SVG styling evolve with the aliViz / BioWidgets codebase. If script and document disagree, treat `tree_clustering.py` as authoritative and update this guide accordingly.

A Colour palette (clusters)

Cluster id $c \geq 0$ uses palette entry $c \bmod 14$. Hex codes below are printed in the corresponding colour, with a swatch:

Index	Hex
0	 #ff00ff
1	 #3b82f6
2	 #10b981
3	 #f59e0b
4	 #8b5cf6
5	 #06b6d4
6	 #84cc16
7	 #f97316
8	 #6366f1
9	 #14b8a6
10	 #64748b
11	 #fbbf24
12	 #a855f7
13	 #22c55e

Group colours start at palette index 1 (group $g \rightarrow \text{index } (g + 1) \bmod 14$). Noise uses  #9ca3af.